

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
		Mr. Bingi Raju	
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week1 – Monday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number:1.3(Present assignment number)/24(Total number of assignments)			
Q.No.	Question		Expected Time to complete
1	Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI Lab Objectives: ❖ To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab. ❖ To understand and use Cursor AI for code generation, explanation, and refactoring. ❖ To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI. ❖ To perform code optimization and documentation using AI tools. Lab Outcomes (LOs): After completing this lab, students will be able to: ❖ Generate Python code using Google Gemini in Google Colab. ❖ Analyze the effectiveness of code explanations and suggestions by Gemini.		Week1 - Monday

	❖ Set up and use Cursor AI for AI-powered coding assistance. ❖ Evaluate and refactor code using Cursor AI features. ❖ Compare AI tool behavior and code quality across different platforms.	
	Task 1: Statistical Summary for Survey Data ❖ Scenario: You are a data analyst intern working with survey responses stored as numerical lists. ❖ Task: Use Google Gemini in Colab to generate a Python function that reads a list of numbers and calculates the mean, minimum, and maximum values. ❖ Code: <pre>Lab 2.1.py > ... 1 #Use Google Gemini in Colab to generate a Python function that reads a list 2 # of numbers and calculates the mean, minimum, and maximum values. Tabnine Edit Test Explain Document 3 def calculate_stats(numbers): 4 if not numbers: 5 return None, None, None 6 mean = sum(numbers) / len(numbers) 7 minimum = min(numbers) 8 maximum = max(numbers) 9 return mean, minimum, maximum 10 numbers = [10, 20, 30, 40, 50] 11 mean, minimum, maximum = calculate_stats(numbers) 12 print("Mean:", mean) 13 print("Minimum:", minimum) 14 print("Maximum:", maximum)</pre> ❖ Expected Output: ➤ Correct Python function ➤ Output shown in Colab ➤ Screenshot of Gemini prompt and result ❖ Output: <pre>PS D:\3-2\Ai assestant coding> & C:/Users/ /Lab 2.1.py" Mean: 30.0 Minimum: 10 Maximum: 50</pre>	
	Task 2: Armstrong Number – AI Comparison ❖ Scenario: You are evaluating AI tools for numeric validation logic. ❖ Task: Generate an Armstrong number checker using Gemini and GitHub Copilot. Compare their outputs, logic style, and clarity. ❖ Code: <pre># Generate an Armstrong number Tabnine Edit Test Explain Document def is_armstrong_number(number): num_str = str(number) num_digits = len(num_str) sum_of_digits = sum(int(digit) ** num_digits for digit in num_str) return number == sum_of_digits number = int(input("Enter a number: ")) if is_armstrong_number(number): print(f"{number} is an Armstrong number.") else: print(f"{number} is not an Armstrong number.")</pre> ❖ Expected Output: ➤ Side-by-side comparison table ➤ Screenshots of prompts and generated code	

	❖ Output: <pre>PS D:\3-2\Ai assestant coding> Enter a number: 153 153 is an Armstrong number.</pre>	
	Task 3: Leap Year Validation Using Cursor AI ❖ Scenario: You are validating a calendar module for a backend system. ❖ Task: Use Cursor AI to generate a Python program that checks whether a given year is a leap year. Use at least two different prompts and observe changes in code. ❖ Code: <pre># Leap Year Tabnine Edit Test Explain Document def is_leap_year(year): if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0): return True else: return False year = int(input("Enter a year: ")) if is_leap_year(year): print(f"{year} is a leap year.") else: print(f"{year} is not a leap year.")</pre> ❖ Expected Output: ➤ Two versions of code ➤ Sample inputs/outputs ➤ Brief comparison ❖ Output: <pre>PS D:\3-2\Ai assestant coding> Enter a year: 2024 2024 is a leap year.</pre>	
	Task 4: Student Logic + AI Refactoring (Odd/Even Sum) ❖ Scenario: Company policy requires developers to write logic before using AI. ❖ Task: Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool. ❖ Code: <pre># Write a Python program that calculates the sum of odd and even numbers in a tuple, then refactor it using any AI tool. Tabnine Edit Test Explain Document def sum_odd_even(numbers): odd_sum = 0 even_sum = 0 for num in numbers: if num % 2 == 0: even_sum += num else: odd_sum += num return odd_sum, even_sum numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9) odd_sum, even_sum = sum_odd_even(numbers) print("Sum of odd numbers:", odd_sum) print("Sum of even numbers:", even_sum)</pre> ❖ Expected Output: ➤ Original code ➤ Refactored code ➤ Explanation of improvements ❖ Output: <pre>PS D:\3-2\Ai assestant coding> Sum of odd numbers: 25 Sum of even numbers: 20</pre>	